

Requerimientos Ágiles / USER STORIES

User Stories es la técnica. ¿Por qué los requerimientos son ágiles? La técnica es la forma de materializar este fundamento conceptual.

Estudiamos User Stories.

Pregunta final: ¿por qué son ágiles los requerimientos ágiles?

La parte práctica del parcial 1 es hacer una user story completa y definir un concepto para generar MVP.

Buscar la página del manifiesto ágil

Filosofía ágil:

No es una metodología.

Se juntaron desarrolladores y llegaron a lo que es el manifiesto ágil.

Es un acuerdo entre más de 1000 firmantes y llegaron a un acuerdo de 4 valores y 12 principios, siendo un compromiso de que c/u de los inventores en que no importan las prácticas de los frameworks de trabajo (el como), debemos de respetar el qué.

Lo que dijeron es que el agilismo es un proceso empírico.

Dentro de los procesos, tenemos 2 tipos:

- los definidos, que necesitamos por comparación. Los estudiamos para comparar.
En los **procesos definidos**, la figura central suele ser un **jefe, director de proyecto o supervisor**, quien tiene la autoridad formal para asignar tareas, controlar el cumplimiento de las etapas y tomar decisiones clave sobre el flujo de trabajo.
- los empíricos: acá, está ágil y lean. Son las 2 filosofías foco y adhieren a este tipo de procesos. Los procesos empíricos son aquellos que funcionan a prueba y error. Los 3 pilares son: inspección (punto de control), adaptación (cambios) y transparencia. El pensamiento ágil adhiere a procesos empíricos, hablando de equipos empoderados, autogestionados

(con capacidad de toma de decisiones). El solo hecho de pertenecer a procesos empíricos trae lo que a estos los define.

Valores ágiles

"Nosotros valoramos más lo que está arriba, que lo que está abajo".



(infografía)

- Valoramos mas individuos y sus interacciones por sobre procesos y herramientas. Para nosotros, es más importante la gente y los vínculos que se establecen. Por ello, se le da poder al equipo para que se autogestione.
- Valoramos más un software en funcionamiento por sobre documentación excesiva/detallada. Lo que realmente le genera valor al cliente, es el software funcionando; por ello, todas nuestras acciones deben apuntar a generar este valor.
- Valoramos más colaborar con el cliente (los FW ágiles arman equipo con él, siendo el Product Owner), que negociaciones contractuales. El PO, que viene del negocio, es parte del equipo.
- Valoramos más la respuesta al cambio por sobre seguir un plan determinado. Con respuesta al cambio, se busca aceptar desde el momento cero que los requerimientos pueden cambiar, (que es contrario a la utopía que definen los procesos definidos, que buscan que los req. se mantengan estables durante el ciclo de vida del producto de SW).

Que se valore algo por sobre otro, no significa que este otro no se efectúe.

Los 12 principios del Manifiesto Ágil (terminar de leerlos)

Dentro de estos 4 valores, se ubican los 12 principios.

1. Nuestra mayor prioridad es satisfacer al cliente (mediante entregas frecuentes de SW funcionando).
2. Aceptar que los requisitos cambien.
3. Entregar software funcional frecuentemente.
4. Los técnicos (diseñadores y desarrolladores) y no técnicos (responsables del negocio) deben trabajar juntos día a día durante el proyecto. El equipo se conforma por perfiles de SW y con perfiles de negocio. En términos de SCRUM, se incorpora el PO. Viene a formar parte del proyecto mientras dura el mismo. Es quien maneja el presupuesto, por lo que las decisiones vinculadas al producto, las tomará él, en términos de qué necesita, qué quiere... (no en términos de cómo, ya que eso es parte del equipo). Se describe con la palabra accountability (responsabilidad, pero es más que ello: es hacerse cargo de las consecuencias de las cosas). El equipo es autogestionado, por lo que no hay un jefe; caso contrario son los procesos definidos (donde sí los hay).
5. Desarrollamos proyectos en torno a individuos motivados.
6. El método más eficiente de comunicar información es cara a cara. Esto es debido a que el cliente es parte del equipo y, en este contexto de transparencia, el PO es tan responsable como el equipo en cuanto al éxito del proyecto.
7. El software funcionando es la principal medida de éxito.
8. Los procesos ágiles promueven el desarrollo sostenible.
9. La atención continua a la excelencia técnica y al buen diseño mejor la Agilidad.
10. La simplicidad es esencial.
11. Las mejores arquitecturas, requerimientos, y diseños **emergen** de equipos auto-organizados. Quien tiene que tomar decisiones es el equipo autogestionado y, por medio de ello, se logran mejores resultados.
12. A intervalos regulares el equipo reflexiona sobre cómo ser más efectivo y de acuerdo a esto ajustan su comportamiento.

¿Qué es Ágil?

NO es una metodología o proceso. Es un pensamiento que define buenas practicas de trabajo y que se usan como base para el equipo.

Ágil es una ideología con un conjunto definido de principios que guían el desarrollo del producto.

Valores de los equipos ágiles ...

- Planificación continua, multi-nivel
- Facultados, auto-organizados, equipos completos
- Entregas frecuentes, iterativas y priorizadas
- Prácticas de ingeniería disciplinadas
- Integración continua
- Testing Concurrente

¿Pero qué significa Ágil?

- Balance entre ningún proceso y demasiado proceso. La diferencia inmediata es la exigencia de una menor cantidad de documentación, sin embargo no es eso lo más importante:
 - Los métodos ágiles son adaptables en lugar de predictivos.
 - Los métodos ágiles son orientados a la gente en lugar de orientados al proceso.

Requerimientos en Agile

Nos referimos a que hay un cambio de concepción del agilismo, en esto de que se le debe dar valor al cliente. Debemos reforzar la idea de que entregar código que compile y sin errores no alcanza, solo sirve si es lo que el cliente necesita y si es lo que tenía en mente cuando pidió el software; este último no es un fin en sí mismo.

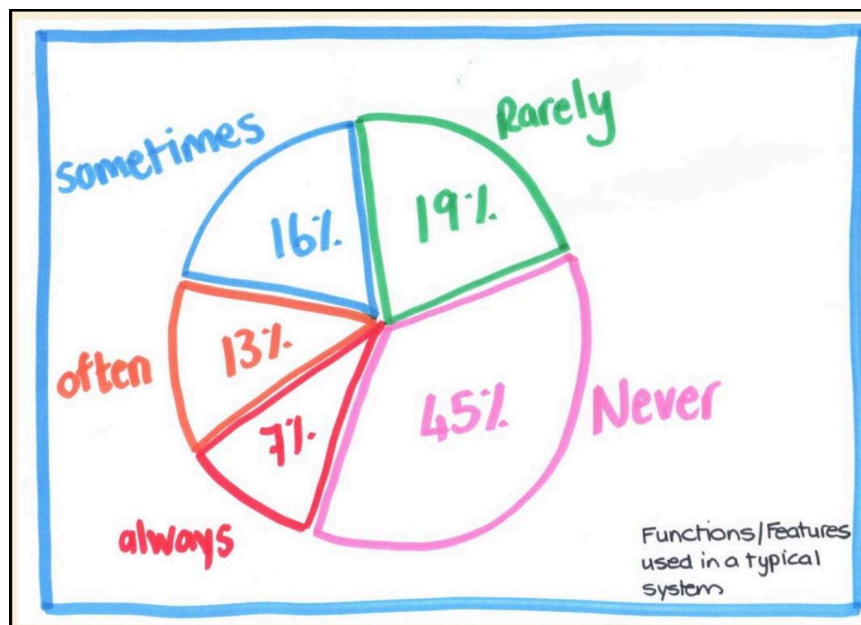
Se busca entender el negocio para poder generar valor para este. Hay una diferencia entre el output (salida de código) y el outcome (resultado/beneficio), que es lo que genera valor para el negocio. El enfoque de requerimientos está enfocado en valor, en resultados. Generar solo código, no alcanza.

Usar el "valor" para construir el producto correcto → Usar historias y modelos para mostrar que construir → Determinar que es "solo lo suficiente" → (vuelta

a empezar)

El costo del tradicional BRUF

Los productos “exitosos” también tienen un desperdicio significativo. Un ejemplo de esto, es el porcentaje de Excel que usamos. La idea es tomar consciencia de esta situación.

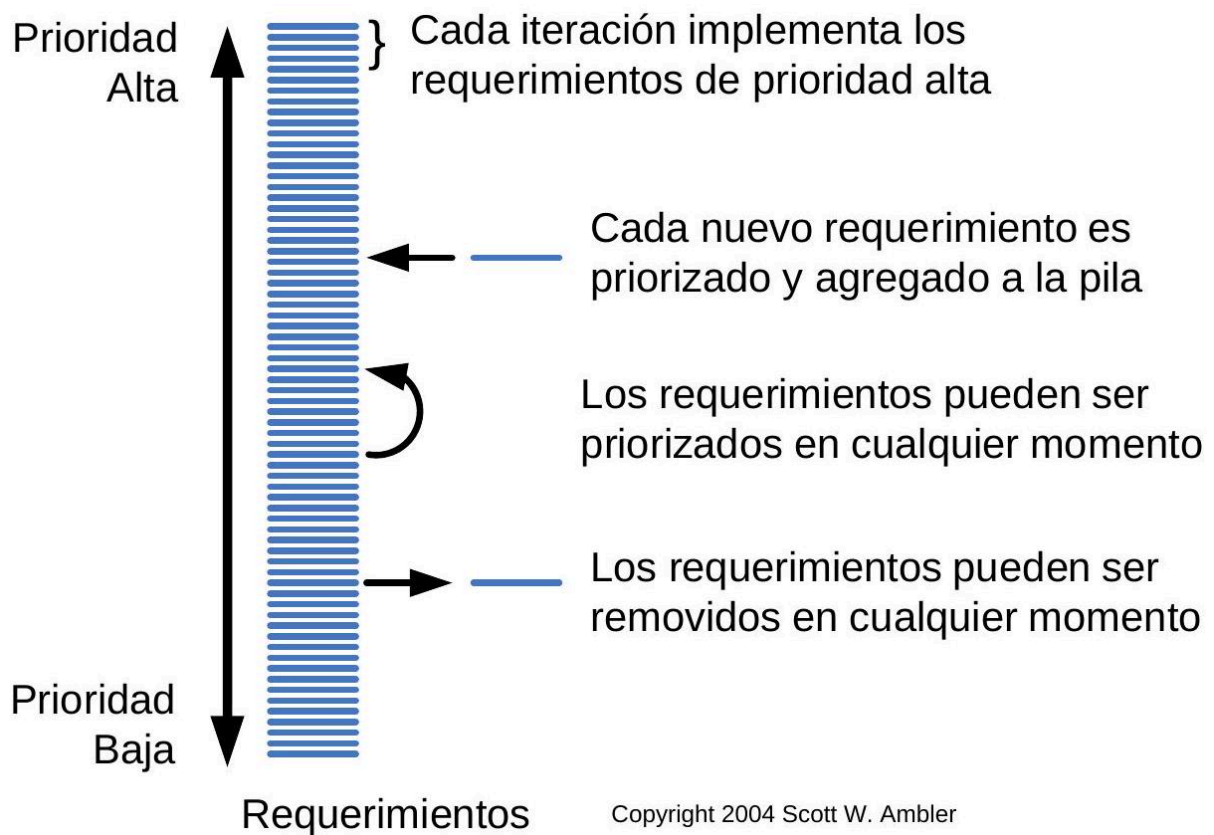


El agilismo dice que no se necesita el 100% del producto especificado, para empezar a producir el producto. No solo que no hace falta, sino que no es posible por la realidad de los requerimientos. Esto es debido a que el 50% de los requerimientos son emergentes (aparecen cuando el cliente empieza a usar el sistema). Tenemos que aceptar esta realidad y comenzar a desarrollar el producto y generar experiencia sobre el mismo, generando una versión rápida, obteniendo feedback, corrigiendo errores y repetir este ciclo.

En cuanto al otro 50%, estamos en condiciones de identificarlo al principio

Gestión Ágil de Requerimientos de Software

Los requisitos cambiantes son una ventaja competitiva si puede actuar sobre ellos.



El cara-a-cara permite que fluya información vocal, subvocal, gestual con realimentación rápida.

El artefacto principal que tiene responsabilidad el PO es el product backlog. Es una lista priorizada de todas las cosas que el detecto que necesita para el producto. El dueño del product backlog es el PO, viniendo del negocio y, aquello que está arriba, tiene mayor prioridad que lo que está abajo.

Cada línea (necesidad) es un ítem de product backlog. Nosotros vamos a usar user stories y, la técnica de user stories, trabaja con diferentes niveles de granularidad para los requerimientos. No vamos a invertir todo el tiempo al principio, trabajando con la idea de "just in time", identificando los requerimientos a medida que los necesitamos para implementarlos. No necesitamos conocer el 100% del producto al principio, ni identificar todos los requerimientos al principio, ya que mucho de ello será desperdicio, por lo que debo convencer al PO de lo que realmente se necesita (lo que está arriba de la pila).

Tipos de Requerimientos

Los casos de uso provienen de requerimientos de software.

Requerimiento de Negocio

Disminuir un X% de tiempo invertido en procesos manuales relacionados con atención al cliente.

Requerimiento de Usuario

Realizar consultas en línea del estado de cuenta de los clientes.
Las user story están aca y vienen del dominio del problema.



Requerimiento Funcional

Generar reporte de saldos de cuenta. Recibir notificaciones por mail.

Requerimiento No funcional

Formato del reporte PDF. Cumplir niveles de seguridad para credenciales de usuarios según la ley bancaria 9999XX

Requerimiento de Implementación

Servidores en la nube

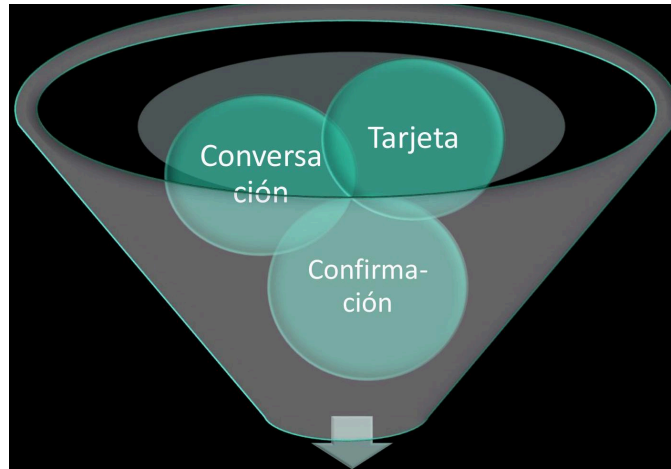
Principios Ágiles relacionados. los Requerimientos Ágiles



User Stories

Se basa en definir una descripción corta de una necesidad del usuario.

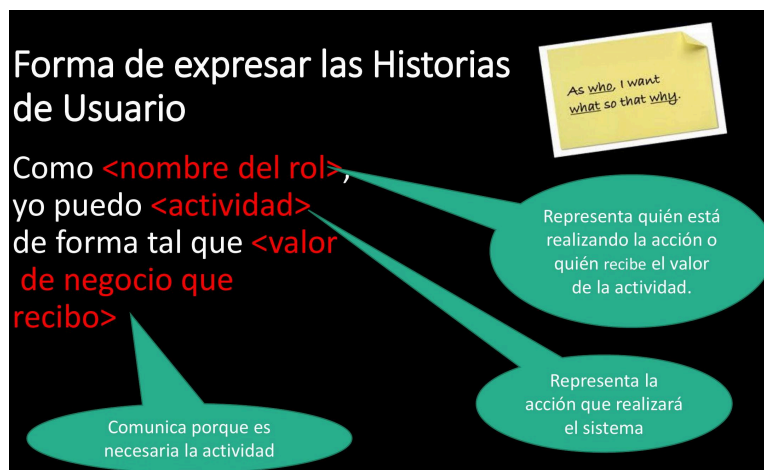
¿Cuáles son las partes de una User Story?



Tienen 3 partes las user stories:

- Tarjeta (card). Es la parte visible de la user story. Tiene 3 partes básicas. Quien necesita lo que se necesita, qué se necesita y para qué se lo necesita
- Conversación (conversación). Es la más importante por el cara-cara.
- Confirmación (confirmation). Son las pruebas de aceptación de usuario. Cómo voy a demostrar que realmente se cumple con lo que el PO necesita.

Formato para las tarjetas:



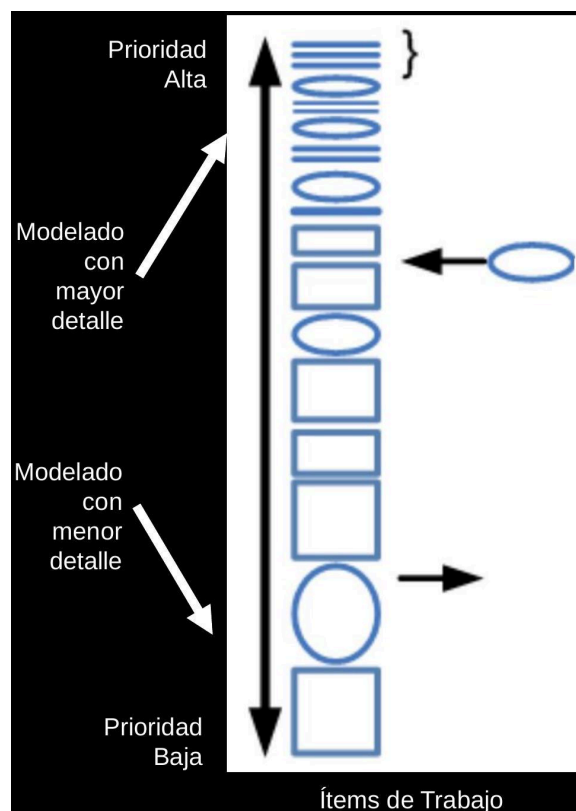
Ejemplo: "como Conductor quiero buscar un destino a partir de una calle y altura para poder llegar al lugar deseado sin perderme".

Las User Stories son Multipropósito

- Las historias son:

- Una necesidad del usuario
- Una descripción del producto
- Un ítem de planificación
- Token para una conversación
- Mecanismo para diferir una conversación

El Product Owner prioriza las historias en el Product Backlog



Cada iteración implementa los ítems de trabajo de mayor prioridad.

Cada nuevo requerimiento es priorizado y agregado a la pila.

Los ítems de trabajos pueden ser repriorizados en cualquier momento.

Los ítems de trabajo pueden ser removidos en cualquier momento.

User stories: Porciones Verticales

Las user stories deben ser porciones verticales. Una user es una funcionalidad que se presenta, pero no se puede entregar únicamente, por ejemplo, la UI o la

base de datos. Debe de ser una porción vertical, con una respectiva UI, servicios, lógica y BD.

Modelado de Roles: Tarjeta de Rol de Usuario

Rol de Usuario: Reclutador Interno

Nos es un experto en computadoras, pero bastante adepto a utilizar la Web. Utilizará el software con poca frecuencia pero muy intensamente. Leerá anuncios de otras compañías para averiguar cuál es la mejor palabra para sus anuncios. La facilidad de uso es importante, pero más importante es que lo que aprenda, lo pueda recordar meses después.

SEARCH AND REPLACE

A user realizes he mis-capitalized a word everywhere in his document, so he tells the word processor to search for all occurrences of it and replace them with the corrected word.

Modelado de Roles: Técnicas Adicionales

- Personas

Mario trabaja como reclutador en el departamento de Speedy Networks, una fábrica de componentes de red de alta gama. El ha trabajado para Speedy Networks por 6 años. Mario tiene un arreglo de horario flexible y trabaja desde casa cada viernes. Mario es muy fuerte con las computadoras y se considera a sí mismo un usuario avanzado de los productos que usa. La esposa de Mario, Kim, está terminando su Doctorado en Química en la Universidad de Stanford. Dado que Speedy Networks ha estado creciendo consistentemente, Mario siempre está buscando ingenieros.

Modelado de Roles: Técnicas Adicionales - Personajes Extremos

Diseño de un PDA para:

- El Papa
- Una mujer de 20 años con muchos novios

- Un traficante de drogas

Tanto la mujer como el traficante desearán mantener agendas separadas en caso de que la vea la policía o un novio. El Papa probablemente tenga menos necesidad de discreción pero querrá un tamaño de fuente más grande.

Usuarios Representantes (Proxies)

Hay un product owner por producto. Si ese PO no puede participar, puede nombrar un representante en su nombre (proxie). Un proxie es cuando el PO ideal no puede involucrarse. SIEMPRE EL PO DEBE DE VENIR DEL NEGOCIO.

- Tipos de usuarios representantes:
 - Gerentes de Usuarios
 - Gerentes de Desarrollo
 - Alguien del grupo de marketing
 - Vendedores
 - Expertos del Dominio
 - Clientes
- Capacitadores y personal de soporte.

Criterios de Aceptación de User Stories

Todas las user van a tener criterios de aceptación. Luego nos van a ayudar a escribir el dorso de la tarjeta.

- Definen límites para una user story (US)
- Ayudan a que los PO respondan lo que necesitan para que la US provea valor (requerimientos funcionales mínimos)
- Ayudan a que el equipo tenga una visión compartida de la US
- Ayudan a desarrolladores y testers a derivar las pruebas.
- Ayudan a los desarrolladores a saber cuando parar de agregar funcionalidad en una US

Ejemplo:

Buscar Destino por Dirección: Como Conductor quiero buscar un destino a partir de una calle y altura para llegar al lugar deseado sin perderme.

Criterios de Aceptación:

- La altura de la calle es un número. Podría ser más específico, como si acepta nulos o no, o que no acepte negativos.
- La búsqueda no puede demorar más de 30 segundos.

¿Cuáles son los Criterios de Aceptación buenos?

- Definen una intención, no una solución
 - Ej.: El usuario debe elegir al menos una cuenta para operar
- Son independientes de la implementación
- Relativamente de alto nivel, no es necesario que se escriba cada detalle

¿Y los detalles?

- Detalles como:
 - El encabezado de la columna se nombra "Saldo"
 - El formato del saldo es 999.999.999,99
 - Debería usarse una lista desplegable en lugar de un Check box.
- Estos detalles que son el resultado de las conversaciones con el PO y el equipo puede capturarlos en dos lugares:
 - Documentación interna de los equipos
 - Pruebas de aceptación automatizadas

Pruebas de Aceptación de User Stories

Front of Card

As a student I want to purchase a parkiz pass so that I can drive to school

Priority: NW Should

Estimate: 4

Copyright 2005-2009 Scott W. Ambler

Back of Card

Confirmations:

The student must pay the correct amount and One pass for one month is issued at a time The student will not receive a pass if the payment isn't sufficient

The person buyiz, the pass most be a comattly enrolled student.
The student my only buy one pass pos month.

Pruebas de Aceptación de Historias de Usuario

Expresan detalles resultantes de la conversación

Complementan la User Story

Proceso de dos pasos:

1. Identificarlas al dorso de la US.
2. Diseñar las pruebas completas

User Story: Tarjeta y Pruebas de Aceptación

Buscar Destino por Dirección

Como Conductor quiero buscar un destino a partir de una calle y altura para poder llegar al lugar deseado sin perderme.

Criterios de Aceptación:

- La altura de la calle es un número.
- La búsqueda no puede demorar más de 30 segundos.

Pruebas de Usuario

- Probar buscar un destino en un país y ciudad existentes, de una calle existente y la altura existente (pasa).
- Probar buscar un destino en un país y ciudad existentes, de una calle inexistente (falla).
- Probar buscar un destino en un país y ciudad existentes, de una calle existente y la altura inexistente (falla).
- Probar buscar un destino en un país inexistente (falla).
- Probar buscar un destino en País existente, ciudad inexistente (falla).
- Probar buscar un destino en un país y ciudad existentes, de una calle existente y demora más de 30 segundos (falla).

Ejemplo: para un software de un GPS

Como Conductor quiero huscar un destino a partir de sus coordenadas para conocer el camino a recorrer para llegar al destino deseado.

Ejemplo:

Como Conductor quiero huscar un destino a partir de sus coordenadas para conocer el camino a recorrer para llegar al destino deseado.

Criterios de Aceptación: Las coordenadas se representan con tres números que indican longitud y tres números que indican latitud. Cada número representa los grados, minutos y segundos respectivamente. Además se debe indicar la orientación (norte, sur, este, oeste).

- Probar buscar un destino en un país y ciudad existentes, de dos coordenadas existentes (pasa).
- Probar huscar un destino en un país y ciudad existentes, de una coordenada inexistente (falla).
- Probar buscar un destino en un país y ciudad existentes, de dos coordenadas existentes sin indicar la orientación (falla).
- Probar ingresar coordenadas de latitud y longitud válidas (pasa).
- Probar ingresar coordenadas de latitud y longitud inválidas (falla).

Ejemplo: User Stories / Casos de Prueba

Como compañía quiero pagar por una búsqueda de puestos con una tarjeta de crédito, así resuelvo mi necesidad en forma más eficiente.

Criterio de Aceptación:

- Se acepta Visa, MasterCard y American Express
- En compras mayores de \$100 se piden el número del dorso de la tarjeta

Probar con Visa (pasa)

Probar con MasterCard (pasa)

Probar con American Express (pasa)

Probar con Dinner's Club (falla)

Probar con números de tarjeta buenos

Probar con números de tarjeta malos

Probar con números de tarjeta faltantes

Probar con tarjetas vencidas

Probar con montos menores de \$100

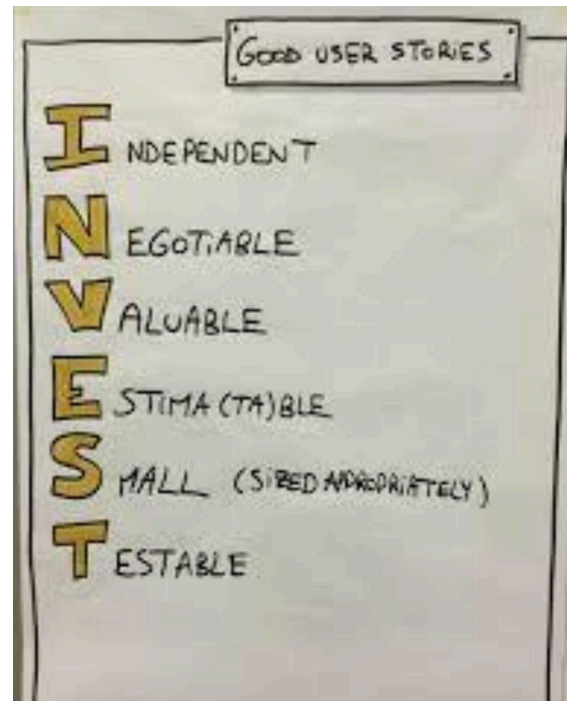
Probar con montos mayores de \$100

Definición de listo - Definition of Ready

Definición de Hecho - Definition of Done

INVEST Model

- Independent - calendarizables e implementables en cualquier orden
- Negotiable - el "qué" no el "cómo". Tal y como vimos anteriormente, el PO decide el "qué", la parte técnica el "cómo".
- Valuable - debe tener valor para el cliente. Es el "para qué", debe tener explícitamente escrito el valor del negocio con la user story. Si tiene valor para el negocio, se la puede priorizar.
- Estimatable - para ayudar al cliente a armar un ranking basado en costos. Tiene que ver en cuanto al tamaño de la user. Cuando tiene un tamaño muy grande, ya no es estimable, por lo que se la pasa a llamar "spike". Una spike es una user story que no se puede estimar (por problemas funcionales o del negocio, o bien un problema técnico). Cuando la incertidumbre es tan grande, la user se la transforma como spike, y va al product backlog; luego, le asigno al equipo tiempo



para que investigue, para así tener menos incertidumbre, logrando que la spike vuelva a ser una user story.

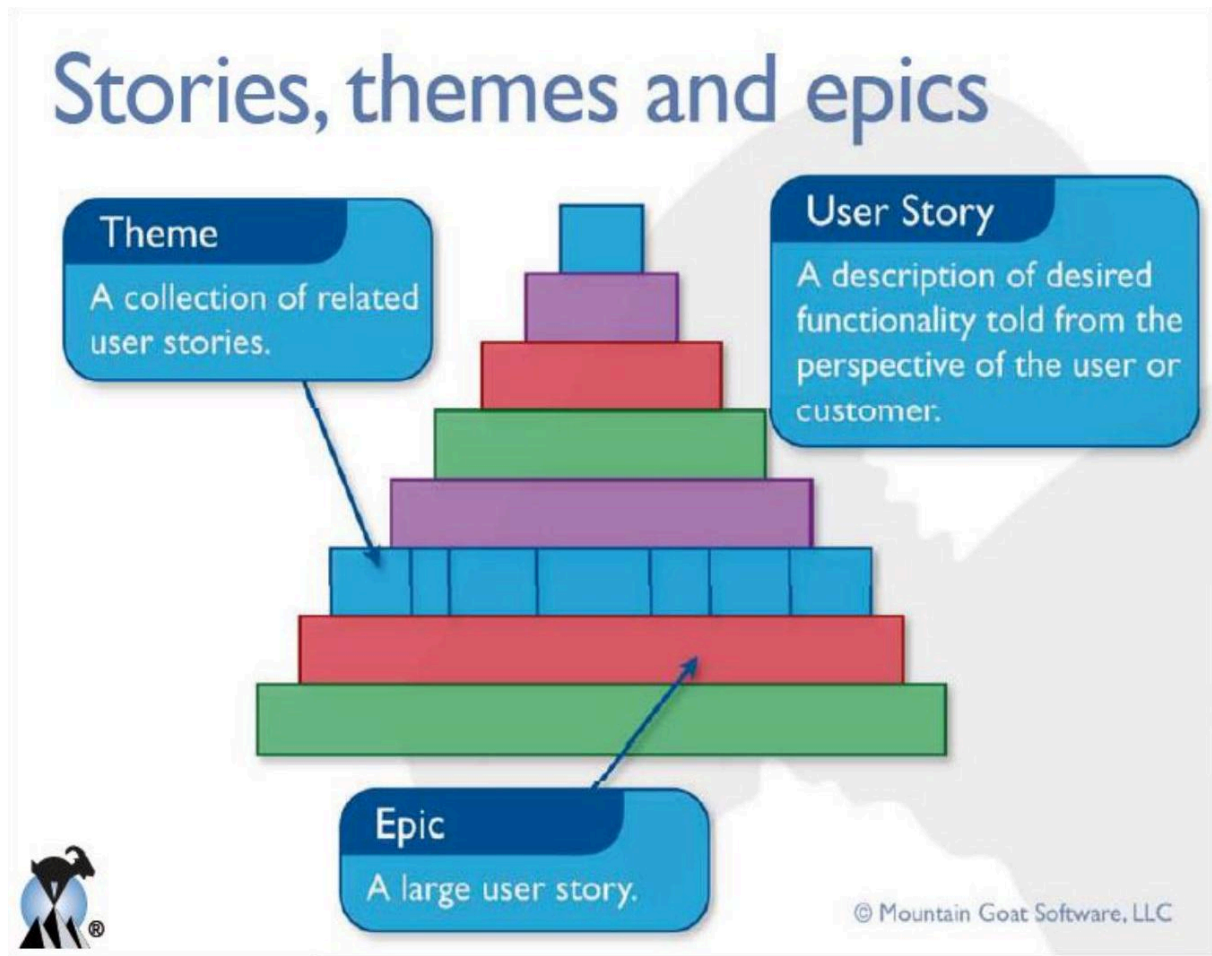
- Small - deben ser "consumidas" en una iteración. Hay que empezarla y terminarla en una iteración, porque el agilismo trabaja con gestión binaria: o esa hecha, o no lo está. El agilismo no gestiona por porcentajes de avance, porque quiere eliminar el síndrome del 90-90.
- Testable - demostrar que fueron implementadas. Sale de los criterios o de la prueba. Tiene que ver con que se le pueda demostrar al PO que está en un servidor de producción para implementarse.

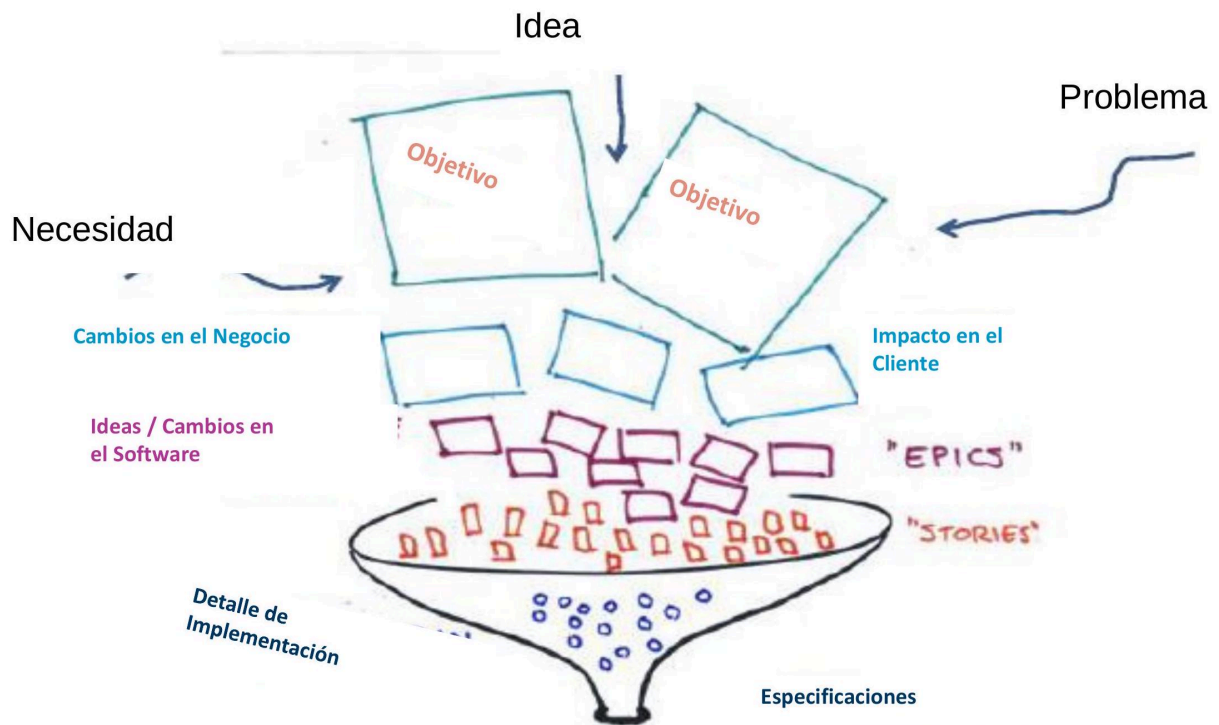
Algo más sobre las User Stories...

- No son especificaciones detalladas de requerimientos (como los casos de uso)
- Son expresiones de intención, "es necesario que haga algo como esto..."
- No están detallados al principio del proyecto, elaborados evitando especificaciones anticipadas, demoras en el desarrollo, inventario de requerimientos y una definición limitada de la solución.
- Necesita poco o nulo mantenimiento y puede descartarse después de la implementación.

- Junto con el código, sirven de entrada a la documentación que se desarrolla incrementalmente después.

Diferentes niveles de abstracción





Spikes

- Tipo especial de historia, utilizado para quitar riesgo e incertidumbre de una User Story u otra faceta del proyecto.
- Se clasifican en : técnicas y funcionales.
- Pueden utilizarse para:
 - Inversión básica para familiarizar al equipo con una nueva tecnología o dominio.
 - Analizar un comportamiento de una historia compleja y poder así dividirla en piezas manejables.
 - Ganar confianza frente a riesgos tecnológicos, investigando o prototipando para ganar confianza.
 - Frente a riesgos funcionales, donde no está claro como el sistema debe resolverla interacción con el usuario para alcanzar el beneficio esperado.

Spikes

Técnicas

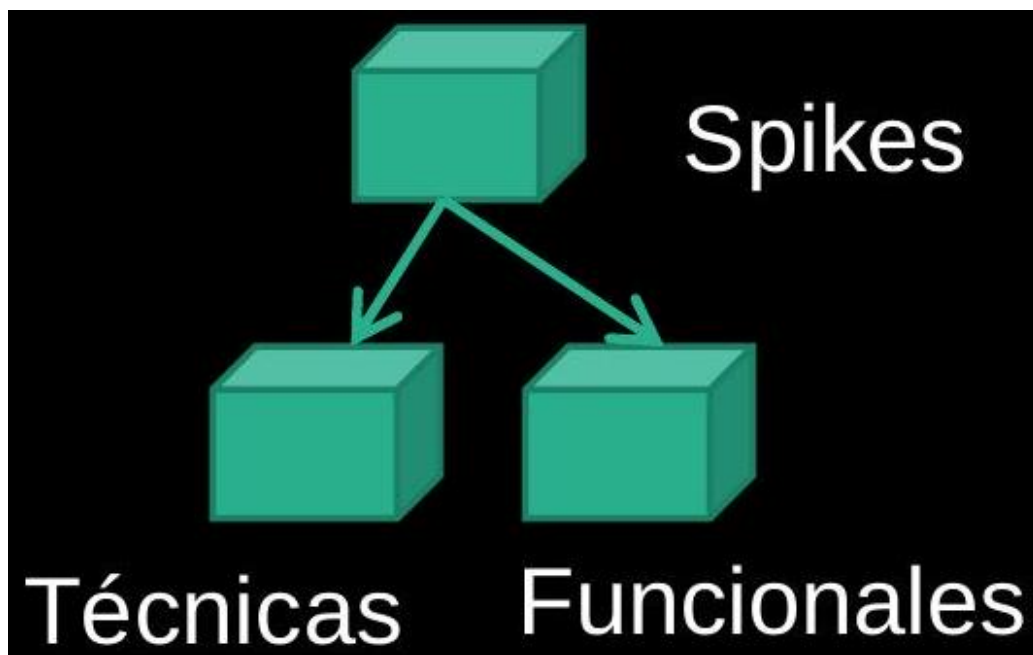
- Utilizadas para investigar enfoques técnicos en el dominio de la solución.

- Evaluar performance potencial
- Decisión hacer o comprar
- Evaluar la implementación de cierta tecnología.
- Cualquier situación en la que el equipo necesite una comprensión más fiable antes de comprometerse a una nueva funcionalidad en un tiempo fijo.

Funcionales

- Utilizadas cuando hay cierta incertidumbre respecto de cómo el usuario interactuará con el sistema.
- Usualmente son mejor evaluadas con prototipos para obtener realimentación de los usuarios o involucrados.

Spikes



Algunas User Stories requieren de ambos tipos de spikes. Por ejemplo:

- Como un cliente, quiero ver mi uso diario de energía en un histograma, para poder comprender rápidamente mi consumo de energía pasado, presente y proyectado.
- En este caso un equipo puede crear dos spikes:
 - Spike Técnico:

- Investigar cuanto tiempo requiere actualizar un display de un cliente al uso actual, determinando requerimientos de comunicación, ancho de banda y si los datos se actualizan en formato push o pull.
- Spike Funcional:
 - Crear un prototipo de histograma en el portal web y obtener la retroalimentación de algunos usuarios respecto del tamaño, el estilo de la presentación y los atributos gráficos.

Lineamientos para Spikes

Estimables, demostrables, y aceptables

La excepción, no la regla

- Toda historia tiene incertidumbre y riesgos.
- El objetivo del equipo es aprender a aceptar y resolver cierta incertidumbre en cada iteración.
- Los spikes deben dejarse para incógnitas mas críticas y grandes.
- Utilizar spikes como última opción.

Implementar la spike en una iteración separada de las historias resultantes

- Salvo que el spike sea pequeño y sencillo y sea probable encontrar una solución rápida en cuyo caso, spike e historia pueden incluirse en la misma iteración.



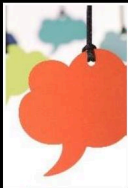
Algunas cosas para dejar en claro

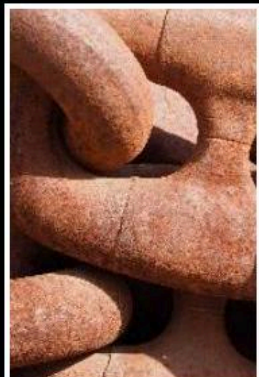
DIFERIR EL ANÁLISIS DETALLADO TAN TARDE COMO SEA POSIBLE, LO QUE ES JUSTO ANTES DE QUE EL TRABAJO COMIENCE.

HASTA ENTONCES, SE CAPTURAN REQUERIMIENTOS EN LA FORMA DE "USER STORIES".

LAS USER STORIES NO SON REQUERIMIENTOS DE SOFTWARE, NO NECESITAN SER DESCRIPCIONES EXHAUSTIVAS DE LA FUNCIONALIDAD DEL SISTEMA.

Tips para que las user stories sean útiles para el equipo

	Un paso a la vez (evitar la palabra “Y”)		Usar palabras claras en los criterios de aceptación
	No olvides la parte invisible: la conversación		Las user stories se escriben desde la perspectiva del usuario

	No forzar todo para escribirlo como user stories
---	--